

Revisão para a A1 — roteiro de estudo

Disciplina: Banco de Dados II

Curso: Bacharelado em Sistemas de Informação

Semestre: 2026.2

Professor: Alysson M. Bruno

Conteúdo revisado: Aulas 01 a 06

Prova A1: 10/10/2026 (sábado)

Turma no Google Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>

Como usar este roteiro

Este roteiro está dividido em **etapas numeradas**. Cada etapa revisa um tema da prova e segue sempre a mesma ordem:

1. **Resumo direto** — o que você precisa lembrar;
2. **Exemplo resolvido** — o conceito aplicado, com a saída real do PostgreSQL;
3. **Armadilhas comuns** — os erros que mais derrubam na prova;
4. **Perguntas de fixação** — responda antes de ler o **Confira a resposta**.

Faça as etapas na ordem e no seu ritmo. Uma etapa por vez já é um bom avanço; não é preciso fazer tudo no mesmo dia. O tempo total estimado é de **cerca de 3 horas**, sem contar a correção do simulado.

Etapas	Tema	Tempo
0	Mapa da prova e autoavaliação	10 min
1	Modelo relacional, integridade e PostgreSQL × MySQL	10 min
2	Normalização, restrições e comandos de escrita	20 min
3	Junções, subconsultas, conjuntos e consultas com defeito	20 min
4	Arquitetura do PostgreSQL e bancos distribuídos	15 min
5	Funções e procedimentos em PL/pgSQL	25 min
6	Controle de fluxo, exceções e cursores	15 min
7	Gatilhos	25 min
8	Simulado — parte objetiva	20 min
9	Simulado — questão discursiva, à mão	20 min
—	Resolução comentada do simulado	depois do simulado

Por onde começar. As Etapas 5, 6 e 7 (funções, procedimentos e gatilhos em PL/pgSQL) são as que mais exigem treino, porque pedem código escrito à mão. Se o seu tempo for curto, comece por elas.

Os exemplos usam uma **pousada hipotética**, criada só para esta revisão, com quatro tabelas. Todo o SQL deste roteiro foi executado no PostgreSQL, e as saídas mostradas são as reais.

```

CREATE TABLE hospede (
  id_hospede bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome       varchar(120) NOT NULL,
  cpf        char(11)     NOT NULL UNIQUE,
  email      varchar(200) UNIQUE
);

CREATE TABLE quarto (
  numero      integer      PRIMARY KEY,
  tipo        varchar(20)  NOT NULL,
  capacidade  integer      NOT NULL CHECK (capacidade > 0),
  diaria      numeric(8,2) NOT NULL CHECK (diaria > 0)
);

CREATE TABLE reserva (
  id_reserva  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_hospede  bigint NOT NULL REFERENCES hospede (id_hospede),
  numero_quarto integer NOT NULL REFERENCES quarto (numero),
  entrada    date      NOT NULL,
  saida      date      NOT NULL,
  pessoas    integer NOT NULL DEFAULT 1 CHECK (pessoas > 0),
  CONSTRAINT saida_depois_da_entrada CHECK (saida > entrada)
);

CREATE TABLE consumo (
  id_consumo  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_reserva  bigint NOT NULL REFERENCES reserva (id_reserva) ON DELETE CASCADE,
  descricao   varchar(60) NOT NULL,
  valor       numeric(8,2) NOT NULL CHECK (valor > 0)
);

```

Os dados de partida:

Tabela	Linhas
hospede	1 Marta Queiroz · 2 Otávio Prado (sem e-mail) · 3 Lívia Moura · 4 Renan Tavares (sem e-mail)
quarto	101 standard (2 pessoas, R\$ 190,00) · 102 standard (2, R\$ 190,00) · 201 família (4, R\$ 330,00) · 202 suíte (2, R\$ 410,00) · 301 família (5, R\$ 350,00)
reserva	1: Marta, quarto 101, 03 a 06/07 · 2: Otávio, 201, 10 a 12/07 · 3: Marta, 202, 01 a 03/08 · 4: Lívia, 101, 15 a 18/08 · 5: Otávio, 102, 20 a 22/06 (todas de 2026)
consumo	reserva 1: jantar R\$ 95,00 e frigobar R\$ 22,50 · reserva 2: passeio de barco R\$ 240,00 · reserva 3: frigobar R\$ 18,00

Repare: o Renan não tem reserva, o quarto 301 nunca foi reservado e as reservas 4 e 5 não têm consumo. Esses “buracos” são de propósito — é neles que as consultas erram. Cada etapa parte desses dados iniciais; as alterações feitas numa etapa não passam para a seguinte.

Etapa 0 — Mapa da prova

Tempo estimado: 10 minutos.

0.1 Como é a prova

Item	Como é
Data	10/10/2026, sábado
Formato	prova escrita, individual e sem consulta , respondida em papel , com caneta
Parte I	10 questões objetivas , com quatro alternativas (A, B, C e D), valendo 0,5 cada — 5,0 pontos
Parte II	1 questão discursiva , valendo 5,0 pontos
Total	10,0 pontos
Conteúdo	Aulas 01 a 06

As objetivas seguem o estilo ENADE: um caso curto, às vezes com código, e uma pergunta. Aparecem três formatos, e todos estão no simulado:

- **alternativa única** — “assinale a opção correta”;
- **afirmações I, II, III e IV** — “é correto apenas o que se afirma em”;
- **asserção e razão** — duas frases ligadas por PORQUE; você julga se cada uma é verdadeira e se a segunda justifica a primeira.

0.2 O que cai

Aula	Tema	Etapa deste roteiro
01	modelo relacional, chaves, integridade, PostgreSQL × MySQL	1
02	normalização (1FN, 2FN e 3FN), restrições, INSERT, UPDATE, DELETE	2
03	junções, subconsultas, operações de conjunto, diagnóstico de consulta com defeito	3
04	arquitetura e organização física do PostgreSQL, bancos distribuídos	4
05	funções e procedimentos em PL/pgSQL	5
06	controle de fluxo, exceções, cursores e gatilhos	6 e 7

0.3 O que não cai

- transações e propriedades ACID como tema (BEGIN, SAVEPOINT, níveis de isolamento);
- concorrência, bloqueios e MVCC;
- índices e planos de execução (EXPLAIN);
- backup e restauração;
- segurança, usuários e permissões;
- as ferramentas usadas nos laboratórios para montar o ambiente.

Atenção a uma exceção: o fato de que **um procedimento pode executar COMMIT e uma função não pode** é conteúdo da Aula 05 e cai (Etapa 5).

0.4 Autoavaliação: por onde começar

Leia cada frase e marque se você **sabe explicar sem olhar**. Cada “não sei” manda você para uma etapa.

Eu sei...	Se não souber, vá para
explicar a diferença entre chave primária, UNIQUE e chave estrangeira	Etapa 1
dizer por que um teste isolado não prova que um SGBD é mais rápido que outro	Etapa 1
apontar se uma tabela viola a 1FN, a 2FN ou a 3FN e como corrigir	Etapa 2
prever se um INSERT, UPDATE ou DELETE será aceito ou recusado	Etapa 2
escolher entre INNER JOIN, LEFT JOIN, FULL OUTER JOIN, UNION, INTERSECT e EXCEPT	Etapa 3
explicar por que NOT IN com NULL devolve zero linhas	Etapa 3
explicar para que servem o WAL, os checkpoints e os buffers compartilhados	Etapa 4
diferenciar fragmentação horizontal, vertical, híbrida e replicação	Etapa 4
dizer quando usar SELECT e quando usar CALL	Etapa 5
explicar por que SUM sem linhas devolve NULL e como o COALESCE resolve	Etapa 5
explicar o que acontece com um CASE sem ELSE e com um laço sem EXIT	Etapa 6
dizer o que o bloco EXCEPTION faz com o erro	Etapa 6
escrever de memória uma função gatilho e o CREATE TRIGGER	Etapa 7
dizer o que NEW vale num DELETE e o que OLD vale num INSERT	Etapa 7
dizer o que devolver (NEW, OLD ou NULL) em cada tipo de gatilho	Etapa 7

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 1 — Modelo relacional, integridade e PostgreSQL × MySQL

Tempo estimado: 10 minutos.

1.1 Resumo direto

- **Banco de dados** é o conjunto organizado de dados. **SGBD** é o software que o gerencia (o PostgreSQL é um SGBD).
- No modelo relacional, a tabela é a **relação**, cada linha é uma **tupla**, cada coluna é um **atributo**, e os valores válidos de uma coluna formam o seu **domínio** — que inclui a regra do negócio, não só o tipo.
- **Chave primária (PK)**: identifica a linha. Não repete e não aceita nulo. Uma por tabela.
- **UNIQUE**: não repete, mas **aceita nulo** (e vários nulos). Pode haver várias.
- **Chave estrangeira (FK)**: o valor precisa existir na tabela referenciada.
- Um relacionamento **muitos-para-muitos** vira uma **tabela associativa**.
- Validar na tela e declarar a regra no banco se **completam**: a tela orienta o usuário; a restrição vale para qualquer caminho de escrita (outro sistema, importação, script).
- **PostgreSQL**: objeto-relacional, desenvolvido por uma comunidade, licença permissiva (pode ser usado em produto comercial), versão principal anual, cerca de cinco anos de suporte por série. Versão **beta** é para teste, não para produção.
- **MySQL**: desenvolvido pela Oracle, edição Community sob GPL, trilhas **LTS** e **Innovation**; também tem MVCC (no InnoDB) e um conjunto amplo de funções JSON.
- **Desempenho não cabe em uma frase**: comparar dois SGBDs exige carga reproduzível, ambiente controlado e análise do plano. Ao relatar um problema, informe a versão, o sistema operacional e a configuração.

1.2 Exemplo resolvido

Tentamos registrar uma reserva para um hóspede que não existe e cadastrar de novo o quarto 101:

```
INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida)
VALUES (99, 101, '2026-09-01', '2026-09-02');
```

```
INSERT INTO quarto (numero, tipo, capacidade, diaria)
VALUES (101, 'standard', 2, 150.00);
```

```
ERROR: insert or update on table "reserva" violates foreign key constraint
"reserva_id_hospede_fkey"
DETAIL: Key (id_hospede)=(99) is not present in table "hospede".
```

```
ERROR: duplicate key value violates unique constraint "quarto_pkey"
DETAIL: Key (numero)=(101) already exists.
```

Agora cadastramos um terceiro hóspede **sem e-mail** (já existem dois assim):

```
INSERT INTO hospede (nome, cpf) VALUES ('Caio Brandão', '100000000005');
```

```
INSERT 0 1
```

Aceito: a coluna email é UNIQUE, mas UNIQUE não obriga o preenchimento, e cada nulo é tratado como diferente dos outros.

1.3 Armadilhas comuns

- Achar que UNIQUE obriga a preencher a coluna. Quem obriga é NOT NULL.
- Achar que a FK impede duplicidade. Ela só garante que o valor **existe** do outro lado; para impedir repetição, use UNIQUE (às vezes composta, com várias colunas).
- Usar nome, telefone ou CPF como chave primária “porque é fácil de lembrar”. Dado com significado pode mudar; o identificador gerado pelo banco (IDENTITY) não muda.
- Dizer que “o PostgreSQL é sempre mais rápido” ou que “o MySQL não tem MVCC”. Nas duas frases, a palavra que derruba é **sempre** (ou **não tem**).
- Confundir objeto-relacional com “orientado a objetos”: o PostgreSQL continua relacional.

1.4 Perguntas de fixação

Pergunta 1. Uma tabela hospede pode ter duas restrições UNIQUE (no CPF e no e-mail) e uma chave primária ao mesmo tempo?

Confira a resposta: pode. Chave primária só existe uma por tabela, mas restrições UNIQUE podem ser várias. É exatamente o que a nossa tabela hospede faz.

Pergunta 2. Um colega rodou a mesma consulta nos dois SGBDs, no notebook dele, e o PostgreSQL foi mais rápido. Isso prova que o PostgreSQL é melhor para a pousada?

Confira a resposta: não. Um teste isolado, sem carga reproduzível nem ambiente controlado, não sustenta conclusão geral. O resultado muda com volume de dados, índices, configuração, versão e hardware.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 2 — Normalização, restrições e comandos de escrita

Tempo estimado: 20 minutos.

2.1 Resumo direto: dependência funcional e formas normais

Dependência funcional $A \rightarrow B$ (“A determina B”): para cada valor de A existe no máximo um valor de B. Ela vem da **regra do negócio**, não dos dados de hoje.

Forma normal	O que exige	Violação típica	Correção
1FN	cada célula tem um valor atômico; sem grupos de colunas repetidas	idiomas = 'português, inglês' ou idioma1, idioma2, idioma3	nova tabela, uma linha por valor
2FN	1FN + nenhum atributo não chave depende de parte de uma chave composta	com chave (A, B), uma coluna depende só de A	a coluna vai para a tabela de A
3FN	2FN + nenhum atributo não chave depende de outro atributo não chave	chave $\rightarrow X \rightarrow Y$ (dependência transitiva)	X e Y vão para uma tabela própria

Duas regras de bolso:

- **Chave simples não tem “parte”**. Se a chave tem uma só coluna, não existe violação de 2FN; o problema, se houver, é de 3FN.
- **“A chave, a chave inteira e nada além da chave.”** Cada coluna não chave deve descrever a chave (1FN), a chave inteira (2FN) e nada além dela (3FN).

2.2 Exemplo resolvido: normalização

Caso 1 — 2FN. A pousada registra a escala das camareiras assim:

```
escala(cpf_camareira, data, andar, camareira_nome)
chave primária: (cpf_camareira, data)
```

camareira_nome depende só de cpf_camareira, que é **parte** da chave. Viola a 2FN. Correção: camereira(cpf, nome) e escala(cpf_camareira, data, andar).

Caso 2 — 3FN. A recepção guardava as reservas numa tabela única:

```
reserva_plana(id_reserva, entrada, hospede_cpf, hospede_nome, numero_quarto,
quarto_tipo, quarto_diaria)
chave primária: id_reserva
```

A chave é simples, então não há 2FN a violar. Mas numero_quarto $\rightarrow$ quarto_tipo, quarto_diaria e hospede_cpf $\rightarrow$ hospede_nome: são dependências **transitivas** (passam por uma coluna que não é chave). Viola a 3FN. Correção: as tabelas quarto e hospede, exatamente como no esquema do início deste roteiro.

2.3 Resumo direto: restrições

Restrição	O que faz	Detalhe que cai
NOT NULL	obriga a ter valor	é ela que obriga, não o UNIQUE
DEFAULT	preenche quando o comando não informa	coluna omitida sem DEFAULT e com NOT NULL dá erro
CHECK	condição sobre a linha	com NULL o resultado é “desconhecido” e a linha passa
PRIMARY KEY	única e não nula	uma por tabela
UNIQUE	não repete	aceita vários nulos
FOREIGN KEY	o valor precisa existir do outro lado	sem ação declarada vale NO ACTION : recusa apagar o pai referenciado

Ações da FK ao apagar o pai: NO ACTION/RESTRICT recusam; CASCADE apaga os filhos junto; SET NULL põe nulo nos filhos (não funciona se a coluna for NOT NULL).

2.4 Exemplo resolvido: restrições e comandos de escrita

Uma pesquisa de satisfação, de 1 a 5 estrelas:

```
CREATE TABLE pesquisa_satisfacao (  
  id_pesquisa bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  id_reserva  bigint NOT NULL REFERENCES reserva (id_reserva),  
  estrelas    integer CHECK (estrelas BETWEEN 1 AND 5)  
);  
  
INSERT INTO pesquisa_satisfacao (id_reserva) VALUES (1);           -- sem  
estrelas  
INSERT INTO pesquisa_satisfacao (id_reserva, estrelas) VALUES (2, 7); -- fora da  
faixa
```

```
INSERT 0 1  
ERROR:  new row for relation "pesquisa_satisfacao" violates check constraint  
"pesquisa_satisfacao_estrelas_check"  
DETAIL:  Failing row contains (2, 2, 7).
```

O primeiro foi aceito com estrelas nulo (o CHECK só reprova quando dá **falso**). O segundo foi recusado.

Um INSERT com várias linhas é tudo ou nada:

```
INSERT INTO pesquisa_satisfacao (id_reserva, estrelas) VALUES (2, 4), (3, 9), (4,  
5);  
SELECT count(*) FROM pesquisa_satisfacao;
```

```
ERROR: new row for relation "pesquisa_satisfacao" violates check constraint
"pesquisa_satisfacao_estrelas_check"
DETAIL: Failing row contains (4, 3, 9).
count
-----
1
```

Só a segunda linha era inválida, mas **nenhuma** das três entrou: a contagem continua 1.

Coluna omitida com DEFAULT e identidade informada à mão:

```
INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida)
VALUES (3, 301, '2026-09-10', '2026-09-12')
RETURNING pessoas;

INSERT INTO hospede (id_hospede, nome, cpf) VALUES (50, 'Teste', '10000000050');
```

```
pessoas
-----
1

ERROR: cannot insert a non-DEFAULT value into column "id_hospede"
DETAIL: Column "id_hospede" is an identity column defined as GENERATED ALWAYS.
HINT: Use OVERRIDING SYSTEM VALUE to override.
```

Apagar o pai referenciado (Marta tem reservas; a FK não declara ação):

```
DELETE FROM hospede WHERE id_hospede = 1;
```

```
ERROR: update or delete on table "hospede" violates foreign key constraint
"reserva_id_hospede_fkey" on table "reserva"
DETAIL: Key (id_hospede)=(1) is still referenced from table "reserva".
```

Já consumo foi declarada com ON DELETE CASCADE: apagar uma reserva sem pesquisa (a reserva 3, que tem um consumo) leva o consumo junto.

```
DELETE FROM reserva WHERE id_reserva = 3;
SELECT count(*) FROM consumo WHERE id_reserva = 3;
```

```
DELETE 1
count
-----
0
```

UPDATE sem WHERE alcança a tabela inteira:

```
UPDATE quarto SET diaria = diaria + 20; -- a intenção era só os quartos
standard
```

```
UPDATE 5
```

Os cinco quartos foram reajustados. O certo seria ... `WHERE tipo = 'standard'`. Hábito seguro: rode antes um `SELECT` com o mesmo `WHERE` e confira quantas linhas voltam.

2.5 Armadilhas comuns

- Achar que chave composta viola a 1FN. Chave com três colunas é permitida.
- Chamar de 2FN uma dependência numa tabela de **chave simples**. Se a chave é uma coluna só, é 3FN.
- Achar que o `CHECK` recusa `NULL`. Não recusa; para obrigar, use `NOT NULL`.
- Achar que num `INSERT` de várias linhas “as boas entram e a ruim fica de fora”.
- Achar que a FK apaga os filhos “por padrão”. O padrão é **recusar** (`NO ACTION`); apagar junto só com `ON DELETE CASCADE`.
- Achar que o PostgreSQL exige `WHERE` no `UPDATE` ou no `DELETE`. Não exige — e é por isso que o `WHERE` esquecido é tão perigoso.

2.6 Perguntas de fixação

Pergunta 1. `hospede(id_hospede, nome, cep, cidade, uf)`, chave `id_hospede`. Regra: o CEP determina a cidade e a UF. Qual forma normal é violada?

Confira a resposta: a 3FN. `cidade` e `uf` dependem de `cep`, que não é chave (dependência transitiva). Não é 2FN, porque a chave é simples. Correção: tabela `cep(cep, cidade, uf)` e `hospede` guardando só o `cep`.

Pergunta 2. `quarto_foto(numero, foto1, foto2, foto3)`. O que está errado?

Confira a resposta: viola a 1FN por **grupo repetido**. O quarto com quatro fotos não cabe. Correção: `foto_quarto(id_foto, numero_quarto, arquivo)`, uma linha por foto.

Pergunta 3. Na tabela `consumo`, a FK para `reserva` fosse declarada `ON DELETE SET NULL`. Apagar uma reserva com consumo daria certo?

Confira a resposta: não. `SET NULL` tentaria pôr nulo em `consumo.id_reserva`, que é `NOT NULL`; o comando seria recusado. `SET NULL` só combina com coluna que aceita nulo.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 3 — Junções, subconsultas, conjuntos e consultas com defeito

Tempo estimado: 20 minutos.

3.1 Resumo direto

Recurso	Devolve
INNER JOIN	só as linhas com par nos dois lados
LEFT JOIN	todas da tabela da esquerda (a do FROM); sem par, a direita vem NULL
RIGHT JOIN	todas da direita (prefira reescrever como LEFT JOIN)
FULL OUTER JOIN	todas dos dois lados, com NULL onde falta par
CROSS JOIN	produto cartesiano: $m \times n$ linhas
self join	a tabela com ela mesma, usando dois apelidos (hierarquias)
UNION	junta os dois resultados e remove repetidas
UNION ALL	junta e mantém repetidas
INTERSECT	só o que está nos dois resultados
EXCEPT	o que está no primeiro e não está no segundo — a ordem importa

Padrões que resolvem perguntas de prova:

- “**Nunca fez / nunca teve**”: LEFT JOIN ... WHERE direita.id IS NULL, ou NOT EXISTS.
- “**Contar, mostrando zero**”: LEFT JOIN + COUNT(coluna_da_direita), nunca COUNT(*).
- “**Acima da média**”: subconsulta escalar no WHERE: WHERE diaria > (SELECT AVG(diaria) FROM quarto).
- **Operação de conjunto**: as duas consultas precisam ter o **mesmo número de colunas**, com tipos compatíveis.

3.2 Exemplo resolvido

Hóspedes que **nunca** reservaram:

```
SELECT h.nome
  FROM hospede h
 LEFT JOIN reserva r ON r.id_hospede = h.id_hospede
 WHERE r.id_reserva IS NULL;
```

nome

Renan Tavares

Reservas por quarto, **mostrando zero**:

```
SELECT q.numero, COUNT(r.id_reserva) AS reservas, COUNT(*) AS linhas
FROM quarto q
LEFT JOIN reserva r ON r.numero_quarto = q.numero
GROUP BY q.numero
ORDER BY q.numero;
```

numero	reservas	linhas
101	2	2
102	1	1
201	1	1
202	1	1
301	0	1

O quarto 301 tem zero reservas, mas COUNT(*) mostra 1: ele conta a linha que o LEFT JOIN completou com NULL.

A ordem do EXCEPT:

```
SELECT numero FROM quarto EXCEPT SELECT numero_quarto FROM reserva; -- quarto
301
SELECT numero_quarto FROM reserva EXCEPT SELECT numero FROM quarto; -- zero
linhas
```

3.3 Diagnóstico de consultas com defeito

Na prova, parte das questões mostra uma consulta que **roda mas dá resultado errado**, e pede a causa e a providência. Estes são os defeitos vistos na Aula 03, todos executados na pousada:

Sintoma	Causa	Providência
NOT IN (subconsulta) devolve zero linhas	a subconsulta devolve algum NULL; x NOT IN (... , NULL) nunca é verdadeiro	NOT EXISTS correlacionado, ou filtrar IS NOT NULL na subconsulta
a contagem veio maior que o esperado	uma junção a mais repetiu cada linha (uma reserva por consumo, por exemplo)	tirar a junção ou usar COUNT(DISTINCT ...)
o total veio menor que o esperado	UNION removeu as linhas repetidas	UNION ALL
more than one row returned by a subquery used as an expression	a subconsulta na lista de colunas devolve várias linhas	filtrar até sobrar no máximo uma linha
each UNION query must have the same number of columns	as duas consultas têm quantidades diferentes de colunas	igualar as colunas
o LEFT JOIN “sumiu” com as linhas sem par	há um filtro sobre a tabela da direita no WHERE	levar o filtro para o ON
as linhas sem par que deviam aparecer não aparecem	a tabela preservada está do lado errado do LEFT JOIN	inverter as tabelas
cada linha veio combinada com todas as outras	CROSS JOIN (ou falta a condição de junção)	INNER JOIN ... ON com a chave

Veja o NOT IN com NULL acontecendo. Uma lista de espera guarda o hóspede 1 e uma linha ainda sem hóspede:

```
CREATE TABLE lista_espera (id serial PRIMARY KEY, id_hospede bigint);
INSERT INTO lista_espera (id_hospede) VALUES (1), (NULL);

SELECT nome FROM hospede
WHERE id_hospede NOT IN (SELECT id_hospede FROM lista_espera);
```

```
nome
-----
(0 rows)
```

Com NOT EXISTS, vêm os três hóspedes que não estão na lista (Otávio, Livia e Renan):

```
SELECT h.nome FROM hospede h
WHERE NOT EXISTS (SELECT 1 FROM lista_espera l WHERE l.id_hospede =
h.id_hospede);
```

E o filtro no lugar errado. Pedido: **todos os quartos**, com as reservas a partir de agosto, se houver.

```
-- com defeito: o filtro sobre r está no WHERE
SELECT q.numero, r.entrada
FROM quarto q
LEFT JOIN reserva r ON r.numero_quarto = q.numero
WHERE r.entrada >= '2026-08-01'
ORDER BY q.numero;
```

numero	entrada
101	2026-08-15
202	2026-08-01

Nos quartos sem reserva em agosto, `r.entrada` é `NULL`, a comparação não dá verdadeiro e o `WHERE` descarta a linha — o `LEFT JOIN` virou, na prática, um `INNER JOIN`. Com o filtro no `ON`, os cinco quartos aparecem:

```
SELECT q.numero, r.entrada
FROM quarto q
LEFT JOIN reserva r ON r.numero_quarto = q.numero
AND r.entrada >= '2026-08-01'
ORDER BY q.numero;
```

numero	entrada
101	2026-08-15
102	
201	
202	2026-08-01
301	

3.4 Armadilhas comuns

- Achar que `LEFT JOIN` “preserva a tabela que tem mais linhas”. Preserva a da **esquerda**, a que vem no `FROM`.
- Escrever `INNER JOIN ... WHERE direita.id IS NULL` para achar ausências: o `INNER JOIN` já descartou quem não tem par; o resultado é vazio.
- Achar que `UNION` “mantém tudo”. Quem mantém é `UNION ALL`.
- Achar que `INTERSECT` junta tudo. Ele só mantém o que é **comum**.
- Usar agregação direto no `WHERE` (`WHERE diaria > AVG(diaria)`): não é permitido; use a subconsulta.

3.5 Perguntas de fixação

Pergunta 1. Quero os números dos quartos que foram reservados **tanto** antes **quanto** depois de 01/08/2026. Qual operação de conjunto uso?

Confira a resposta: `INTERSECT`, que devolve só o que aparece nos dois resultados. Na pousada, o resultado é o quarto 101 (reservas em julho e em agosto).

Pergunta 2. Um relatório mostra, para cada hóspede, a quantidade de reservas, mas junta também a tabela consumo. A Marta tem 2 reservas e o relatório mostrou 3. Por quê?

Confira a resposta: a junção com consumo repete a reserva uma vez por consumo (a reserva 1 tem dois). Executado na pousada, `COUNT(r.id_reserva)` deu 3 e `COUNT(DISTINCT r.id_reserva)` deu 2. Providência: tirar a junção ou usar `DISTINCT`.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 4 — Arquitetura do PostgreSQL e bancos distribuídos

Tempo estimado: 15 minutos.

4.1 Resumo direto: o servidor por dentro

Peça	O que lembrar
cliente e servidor	o programa cliente (o <code>psql</code> , o <code>pgAdmin</code> , o sistema da pousada) só envia SQL; as tabelas ficam no servidor
caminho do comando	o servidor analisa o comando, o planejador escolhe a estratégia e o executor acessa os dados
processos	cada conexão é atendida por um processo de backend próprio; outros processos cuidam da gravação, do WAL e dos checkpoints
buffers compartilhados	memória com páginas usadas há pouco; por isso a segunda execução de uma consulta pode ser mais rápida
páginas	tabelas e índices são divididos em páginas de 8 KiB (tamanho padrão)
organização lógica	servidor → banco → esquema → tabela; o esquema é um espaço de nomes (não é um disco)
catálogo	guarda os metadados (tabelas, colunas, tipos, usuários, rotinas), não os dados das suas tabelas
arquivos	<code>pg_relation_filepath</code> mostra onde fica o arquivo da tabela — só para observar ; editar o arquivo pode corromper o banco
WAL	cada alteração é registrada no WAL antes de ir para as páginas de dados; depois de uma queda, o servidor usa o WAL para refazer o que já estava confirmado
checkpoint	ponto de sincronização que limita o trecho de WAL a reprocessar na recuperação
WAL e backup	WAL não substitui backup : não protege de exclusão por engano percebida tarde, nem da perda do servidor inteiro

4.2 Resumo direto: bancos distribuídos

Arquitetura ou técnica	Como reconhecer
centralizada / cliente-servidor	vários clientes, um servidor com os dados — clientes em cidades diferentes não tornam o banco distribuído
banco distribuído	dados logicamente relacionados guardados em mais de um nó
fragmentação horizontal	separa linhas por um critério (unidade, região, período); reconstrói por união
fragmentação vertical	separa colunas ; a chave fica nos dois fragmentos; reconstrói por junção pela chave
fragmentação híbrida	as duas ao mesmo tempo (por exemplo, linhas por unidade e, dentro de cada unidade, colunas)
replicação	cópias dos mesmos dados em vários nós; exige manter as cópias coerentes
transparência	o usuário consulta a relação lógica sem saber em qual nó está a linha (localização), como ela foi dividida (fragmentação) ou qual cópia foi usada (replicação)

Uma fragmentação bem feita **cobre todas as linhas** e **não inventa nem perde dados** ao ser reconstruída.

4.3 Exemplo resolvido

```
SHOW block_size;
```

```
block_size
-----
8192
```

8192 bytes = 8 KiB, o tamanho da página.

Dois esquemas com uma tabela de mesmo nome convivem no mesmo banco, porque o esquema é um espaço de nomes:

```
CREATE SCHEMA recepcao;
CREATE SCHEMA financeiro;
CREATE TABLE recepcao.pagamento (id integer PRIMARY KEY, obs text);
CREATE TABLE financeiro.pagamento (id integer PRIMARY KEY, valor numeric(8,2));

SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_name = 'pagamento'
ORDER BY table_schema;
```

```

table_schema | table_name
-----+-----
financeiro   | pagamento
recepcao     | pagamento

```

A consulta a `information_schema.tables` lê o **catálogo**: são metadados sobre as tabelas.

Classificando propostas. A pousada tem uma unidade no lago e outra na serra.

Proposta	Classificação
cada unidade guarda só as suas reservas, com todas as colunas	fragmentação horizontal
os dados de contato do hóspede num servidor e os dados de pagamento em outro, com <code>id_hospede</code> nos dois	fragmentação vertical
cada unidade guarda uma cópia completa da tabela quarto	replicação
as duas unidades acessam, pela internet, um único servidor	cliente-servidor com dados centralizados

4.4 Armadilhas comuns

- Achar que a tabela “fica no pgAdmin”. Fica no servidor.
- Achar que o WAL dispensa o backup.
- Achar que o esquema é um disco ou uma pasta separada.
- Achar que clientes em várias cidades fazem um banco distribuído.
- Reconstruir fragmento vertical com `UNION` ou horizontal com junção. É o contrário.
- Numa asserção-razão, marcar “a II justifica a I” só porque as duas são verdadeiras. Pergunte: **a II explica a I?** Duas frases verdadeiras sobre assuntos diferentes não formam uma relação de causa.

4.5 Perguntas de fixação

Pergunta 1. Na sexta-feira, alguém apagou por engano todos os consumos da pousada, e isso só foi notado na segunda. O WAL do PostgreSQL resolve o problema?

Confira a resposta: não. O WAL serve para o servidor refazer, depois de uma falha, as alterações que já estavam confirmadas — e a exclusão também estava confirmada. Para voltar ao estado anterior, é preciso **backup**. WAL não substitui backup.

Pergunta 2. Dividir `hospede` em `hospede_contato(id_hospede, nome, email)` e `hospede_pagamento(id_hospede, cartao_final, banco)`. Que fragmentação é essa e como se reconstrói?

Confira a resposta: fragmentação **vertical** (separa colunas). Reconstrói-se por **junção** pela chave

id_hospede, que fica nos dois fragmentos.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 5 — Funções e procedimentos em PL/pgSQL

Tempo estimado: 25 minutos. Esta etapa pesa muito na prova.

5.1 Resumo direto: a anatomia

```
CREATE OR REPLACE FUNCTION nome(p_parametro tipo)
RETURNS tipo_do_resultado      -- função: obrigatório
LANGUAGE plpgsql                -- ou sql
AS $$
DECLARE                        -- opcional: só se houver variáveis
    v_variavel tipo;
BEGIN                          -- início do bloco (NÃO abre transação)
    ...
    RETURN valor;              -- função que devolve valor: obrigatório
END;
$$;
```

Convenções da disciplina: fn_ função, sp_ procedimento, p_ parâmetro, v_ variável, c_ cursor, fn_tg_ função gatilho. Os prefixos p_ e v_ evitam a ambiguidade entre variável e coluna.

	Função	Procedimento
Criação	CREATE FUNCTION ... RETURNS tipo	CREATE PROCEDURE, sem RETURNS
Chamada	dentro de um comando: SELECT fn_x(1);, na lista de colunas, no WHERE, no FROM	comando próprio: CALL sp_x(1);
Devolve	sempre um resultado (valor ou tabela)	nada, salvo por parâmetro INOUT
COMMIT no corpo	não pode	pode
Ligado a gatilho	sim (a função gatilho)	não

Mais pontos que caem:

- **SELECT ... INTO v_x** guarda o resultado de uma consulta na variável. Se a consulta não trazer **nenhuma linha**, a variável fica NULL — **sem erro**.
- **SUM de nenhuma linha é NULL**, não zero. **COALESCE(SUM(...), 0)** troca o NULL por zero.
- **Valor-padrão:** p_percentual numeric DEFAULT 15. Se o argumento for omitido, vale o padrão; se for informado, vale o informado. Parâmetros com padrão vêm **depois** dos sem padrão.
- **INOUT** faz o CALL devolver uma linha com o valor final do parâmetro; com INSERT ... RETURNING id INTO p_id, o procedimento devolve o id gerado.

- **RETURNS TABLE (...)**: a função é chamada no FROM e produz as linhas com RETURN QUERY. Os nomes do RETURNS TABLE viram variáveis: **qualifique as colunas** com o apelido da tabela.
- **LANGUAGE sql** basta quando o corpo é uma única consulta; não tem variáveis, IF nem EXCEPTION.
- Volatilidade: IMMUTABLE (só depende dos argumentos), STABLE (lê tabelas, não altera), VOLATILE (padrão).
- CREATE OR REPLACE **não troca o tipo de retorno**: é preciso DROP FUNCTION antes.
- O código da rotina fica no **catálogo** do banco (pg_proc), não num arquivo do cliente.
- **Sobrecarga**: duas rotinas podem ter o mesmo nome com parâmetros diferentes; por isso o DROP FUNCTION pode exigir os tipos: DROP FUNCTION fn_x(integer, date);.

5.2 Exemplo resolvido: SUM vazio e COALESCE

Total de consumo de uma reserva, **primeira versão**:

```
CREATE OR REPLACE FUNCTION fn_total_consumo(p_reserva_id bigint)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    v_total numeric;
BEGIN
    SELECT SUM(valor) INTO v_total
    FROM consumo
    WHERE id_reserva = p_reserva_id;
    RETURN v_total;
END;
$$;

SELECT fn_total_consumo(1) AS reserva_1, fn_total_consumo(4) AS reserva_4;
```

```
reserva_1 | reserva_4
-----+-----
    117.50 |
```

A reserva 1 soma $95,00 + 22,50 = 117,50$. A reserva 4 **não tem consumo**: o SUM de nenhuma linha é NULL, e a coluna aparece vazia. Trocando a linha do SELECT por SELECT COALESCE(SUM(valor), 0) INTO v_total:

```
reserva_1 | reserva_4
-----+-----
    117.50 |          0
```

A função é avaliada **linha a linha** quando aparece na lista de colunas:

```
SELECT id_reserva, fn_total_consumo(id_reserva) AS consumo
FROM reserva ORDER BY id_reserva;
```

id_reserva	consumo
1	117.50
2	240.00
3	18.00
4	0
5	0

5.3 Exemplo resolvido: SELECT × CALL, RETURN e valor-padrão

Chamar função com CALL e procedimento com SELECT dá erro, e a própria mensagem ensina:

```
ERROR: fn_total_consumo(integer) is not a procedure
HINT: To call a function, use SELECT.
```

```
ERROR: sp_lancar_consumo(integer, unknown, numeric, unknown) is a procedure
HINT: To call a procedure, use CALL.
```

Valor-padrão:

```
CREATE OR REPLACE FUNCTION fn_diaria_promocional(
    p_diaria      numeric,
    p_percentual  numeric DEFAULT 15
)
RETURNS numeric
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN round(p_diaria - p_diaria * p_percentual / 100, 2);
END;
$$;

SELECT fn_diaria_promocional(240.00) AS padrao,
       fn_diaria_promocional(240.00, 30) AS informado;
```

padrao	informado
204.00	168.00

Caminho sem RETURN: uma função que só tem RETURN dentro do IF funciona quando a condição é verdadeira e falha quando não é.

```

CREATE OR REPLACE FUNCTION fn_categoria_estadia(p_noites integer)
RETURNS text
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_noites >= 7 THEN
        RETURN 'longa';
    ELSIF p_noites >= 3 THEN
        RETURN 'média';
    END IF;
END;
$$;

SELECT fn_categoria_estadia(10);
SELECT fn_categoria_estadia(2);

```

```

fn_categoria_estadia
-----
longa

ERROR:  control reached end of function without RETURN
CONTEXT:  PL/pgSQL function fn_categoria_estadia(integer)

```

Não devolve NULL: dá **erro**. A correção é um ELSE RETURN 'curta';.

5.4 Exemplo resolvido: procedimento com INOUT

```

CREATE OR REPLACE PROCEDURE sp_lancar_consumo(
    p_reserva_id      bigint,
    p_descricao       text,
    p_valor           numeric,
    INOUT p_consumo_id bigint
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO consumo (id_reserva, descricao, valor)
    VALUES (p_reserva_id, p_descricao, p_valor)
    RETURNING id_consumo INTO p_consumo_id;
END;
$$;

CALL sp_lancar_consumo(4, 'lavanderia', 35.00, NULL);

```

```

p_consumo_id
-----
5

```

O CALL devolveu uma linha com o id gerado (já havia quatro consumos). E uma **função** que tenta COMMIT:

```
ERROR: invalid transaction termination
CONTEXT: PL/pgSQL function fn_tenta_confirmar() line 4 at COMMIT
```

5.5 Exemplo resolvido: RETURNS TABLE e a ambiguidade

```
CREATE OR REPLACE FUNCTION fn_reservas_do_hospede(p_hospede_id bigint)
RETURNS TABLE (numero_quarto integer, entrada date, noites integer)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT numero_quarto, entrada, saida - entrada
           FROM reserva
          WHERE id_hospede = p_hospede_id;
END;
$$;

SELECT * FROM fn_reservas_do_hospede(1);
```

```
ERROR: column reference "numero_quarto" is ambiguous
DETAIL: It could refer to either a PL/pgSQL variable or a table column.
```

numero_quarto é, ao mesmo tempo, coluna de reserva e variável criada pelo RETURNS TABLE. Qualificando com o apelido r:

```
RETURN QUERY
    SELECT r.numero_quarto, r.entrada, r.saida - r.entrada
       FROM reserva r
      WHERE r.id_hospede = p_hospede_id
      ORDER BY r.entrada;
```

numero_quarto	entrada	noites
101	2026-07-03	3
202	2026-08-01	2

5.6 Armadilhas comuns

- SELECT sp_... ou CALL fn_...: trocar a forma de chamar dá erro.
- Achar que o BEGIN do bloco PL/pgSQL abre uma transação. Não abre.
- Achar que DECLARE é obrigatório. Só é necessário quando há variáveis.
- Achar que função sem RETURN “devolve NULL”. Em PL/pgSQL, a função que devolve valor e chega ao END sem RETURN **dá erro**.
- Achar que o valor-padrão vale “sempre”. Ele só vale quando o argumento é omitido.

- Esquecer que SUM sem linhas é NULL — e que COUNT sem linhas é 0.
- Achar que SELECT ... INTO sem linha dá erro. A variável fica NULL.
- Chamar função RETURNS TABLE com CALL. Ela vai no FROM.

5.7 Perguntas de fixação

Pergunta 1. SELECT fn_diaria_promocional(250.00, 20); — quanto devolve?

Confira a resposta: 200.00. O segundo argumento foi informado (20), então o padrão de 15 não é usado: $250 - 250 \times 20 / 100 = 200$.

Pergunta 2. Por que fn_total_consumo deve ser declarada STABLE, e não IMMUTABLE?

Confira a resposta: porque ela **lê uma tabela**: com o mesmo argumento, o resultado muda quando alguém lança um consumo. IMMUTABLE é só para quem depende apenas dos argumentos, como um cálculo de desconto.

Pergunta 3. Por que fn_tenta_confirmar, da seção 5.4, falhou com invalid transaction termination? Que objeto permitiria o COMMIT?

Confira a resposta: porque uma **função** é avaliada dentro do comando que a chamou e não pode encerrar a transação. Só um **procedimento**, chamado com CALL, pode executar COMMIT no próprio corpo.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 6 — Controle de fluxo, exceções e cursores

Tempo estimado: 15 minutos.

6.1 Resumo direto

Recurso	O que lembrar
IF ... ELSIF ... ELSE ... END IF;	a palavra é ELSIF ; o primeiro THEN verdadeiro vence e os seguintes nem são testados
CASE ... END CASE;	sem ELSE, quando nenhum WHEN casa: ERROR: case not found
LOOP ... END LOOP;	repete para sempre; precisa de EXIT ou EXIT WHEN
WHILE cond LOOP	testa antes de cada volta
FOR i IN 1..n LOOP	a variável i não é declarada
FOR v_linha IN SELECT ... LOOP	cursor implícito: abre, percorre e fecha sozinho
FOUND	verdadeiro se o último SELECT INTO, UPDATE, DELETE ou FETCH achou linha
RAISE NOTICE	mostra um aviso e continua
RAISE EXCEPTION	mostra a mensagem e interrompe ; nada do comando fica gravado
USING HINT = '...'	orienta quem lê o erro
bloco EXCEPTION WHEN ... THEN	captura o erro: ele não sobe para quem chamou, o que o bloco alterou é desfeito e a execução segue pelo tratador
SQLSTATE / SQLERRM	código (ex.: 23505) e mensagem do erro, dentro do tratador
cursor explícito	DECLARE c CURSOR FOR → OPEN → FETCH ... INTO → EXIT WHEN NOT FOUND → CLOSE

Condições de erro mais usadas: division_by_zero, unique_violation, foreign_key_violation, not_null_violation, check_violation, OTHERS.

6.2 Exemplo resolvido: a ordem do IF

```
-- versão certa: do maior para o menor
IF p_noites >= 7 THEN RETURN 'longa';
ELSIF p_noites >= 3 THEN RETURN 'média';
ELSE RETURN 'curta';
END IF;
```

Com essa ordem, 10, 3 e 1 noite dão longa, média e curta. **Invertendo** as duas primeiras condições (>= 3 antes de >= 7), a chamada com 10 noites devolveu média: 10 >= 3 já é verdadeiro e o ELSIF nunca é testado. Nenhum erro aparece — só o resultado errado.

6.3 Exemplo resolvido: FOUND, RAISE EXCEPTION e HINT

```
CREATE OR REPLACE PROCEDURE sp_trocar_quarto(p_reserva_id bigint, p_novo_quarto
integer)
LANGUAGE plpgsql
AS $$
DECLARE
    v_capacidade integer;
BEGIN
    SELECT capacidade INTO v_capacidade FROM quarto WHERE numero = p_novo_quarto;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Quarto % não existe.', p_novo_quarto
            USING HINT = 'Confira a lista com SELECT numero FROM quarto;';
    END IF;

    UPDATE reserva SET numero_quarto = p_novo_quarto WHERE id_reserva =
p_reserva_id;
    RAISE NOTICE 'Reserva % transferida para o quarto %.', p_reserva_id,
p_novo_quarto;
END;
$$;

CALL sp_trocar_quarto(5, 999);
```

```
ERROR:  Quarto 999 não existe.
HINT:   Confira a lista com SELECT numero FROM quarto;
CONTEXT: PL/pgSQL function sp_trocar_quarto(bigint,integer) line 8 at RAISE
```

A reserva 5 continuou no quarto 102: o RAISE EXCEPTION parou a rotina **antes** do UPDATE. Leia as três linhas: ERROR (o quê), HINT (o que fazer) e CONTEXT (onde).

6.4 Exemplo resolvido: capturar o erro com EXCEPTION

```
CREATE OR REPLACE PROCEDURE sp_cadastrar_hospede(p_nome text, p_cpf char(11))
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO hospede (nome, cpf) VALUES (p_nome, p_cpf);
    RAISE NOTICE 'Hóspede % cadastrado.', p_nome;
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'O CPF % já está cadastrado.', p_cpf;
        RAISE NOTICE 'Código: % - %', SQLSTATE, SQLERRM;
END;
$$;

CALL sp_cadastrar_hospede('Paula Siqueira', '10000000009');
CALL sp_cadastrar_hospede('Outra Paula', '10000000009');
```

```
NOTICE:  Hóspede Paula Siqueira cadastrado.
CALL
NOTICE:  O CPF 100000000009 já está cadastrado.
NOTICE:  Código: 23505 – duplicate key value violates unique constraint
"hospede_cpf_key"
CALL
```

A segunda chamada terminou com CALL, sem erro: o tratador **engoliu** a falha. Nada foi gravado, mas quem chamou não recebeu erro nenhum. Capturar é uma decisão de projeto — e capturar tudo com WHEN OTHERS sem registrar nada esconde defeitos.

6.5 Exemplo resolvido: cursor explícito

```
CREATE OR REPLACE PROCEDURE sp_listar_quartos_livres(p_tipo text)
LANGUAGE plpgsql
AS $$
DECLARE
    c_quarto CURSOR (p_t text) FOR
        SELECT q.numero, q.diaria
        FROM quarto q
        WHERE q.tipo = p_t
        AND NOT EXISTS (SELECT 1 FROM reserva r WHERE r.numero_quarto =
q.numero)
        ORDER BY q.numero;
    v_linha record;
BEGIN
    OPEN c_quarto(p_tipo);
    LOOP
        FETCH c_quarto INTO v_linha;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Quarto % livre, diária R$ %', v_linha.numero,
v_linha.diaria;
    END LOOP;
    CLOSE c_quarto;
END;
$$;

CALL sp_listar_quartos_livres('família');
```

```
NOTICE:  Quarto 301 livre, diária R$ 350.00
CALL
```

Sem a linha EXIT WHEN NOT FOUND, o FETCH para de trazer linhas, mas o LOOP continua girando: a sessão não termina. **Regra de bolso:** se dá para fazer com um comando SQL de conjunto (um UPDATE, um SELECT), prefira o comando ao cursor.

6.6 Armadilhas comuns

- Escrever ELSEIF ou ELSE IF. É ELSIF.

- Achar que o IF “escolhe a condição mais específica”. Ele pega a **primeira** verdadeira.
- Esquecer o ELSE do CASE (erro case not found).
- Esquecer o EXIT WHEN NOT FOUND no cursor explícito (laço sem fim).
- Achar que o bloco EXCEPTION “desliga” a restrição. A restrição continua valendo; o tratador só decide o que fazer depois que ela recusou.
- Achar que RAISE NOTICE interrompe a rotina. Só RAISE EXCEPTION interrompe.

6.7 Perguntas de fixação

Pergunta 1. Um CASE p_tipo WHEN 'S' THEN ... WHEN 'F' THEN ... END CASE; recebe 'X'. O que acontece?

Confira a resposta: erro. Executado: ERROR: case not found, com a dica CASE statement is missing ELSE part. Correção: acrescentar um ELSE.

Pergunta 2. Em sp_cadastrar_hospede (seção 6.4), troque o tratador por WHEN OTHERS THEN NULL;. O que acontece quando o CPF se repete? Por que isso é perigoso?

Confira a resposta: nada é gravado, nenhum aviso aparece e quem chamou não recebe erro: o CALL termina como se tudo tivesse dado certo. É perigoso porque qualquer falha, inclusive um defeito de programação, passa a ser escondida em silêncio.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 7 — Gatilhos

Tempo estimado: 25 minutos. **Faça esta etapa com calma e escreva o código à mão.**

7.1 Resumo direto: as duas peças

Um gatilho é uma regra que o **próprio servidor dispara** quando a tabela sofre um evento. Ninguém precisa lembrar de chamá-lo. São sempre **duas peças**:

```
-- Peça 1: a função gatilho
CREATE OR REPLACE FUNCTION fn_tg_nome()
RETURNS trigger          -- tipo trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF <regra violada> THEN
        RAISE EXCEPTION 'mensagem clara';
    END IF;
    RETURN NEW;           -- ou OLD, ou NULL (veja 7.3)
END;
$$;

-- Peça 2: o comando que liga a função à tabela
CREATE TRIGGER tg_nome
    BEFORE INSERT OR UPDATE ON tabela
    FOR EACH ROW
    EXECUTE FUNCTION fn_tg_nome();
```

A função gatilho **não tem parâmetros** e devolve o tipo **trigger**. Os erros quando isso não é respeitado:

```
ERROR: trigger functions cannot have declared arguments
ERROR: function fn_nao_trigger must return type trigger
```

7.2 Resumo direto: OLD, NEW e as outras variáveis

operação	OLD	NEW
INSERT	NULL não há linha antiga · sem erro	a linha que está entrando
UPDATE	a linha como estava	a linha como vai ficar
DELETE	a linha que está saindo	NULL não há linha nova · sem erro

Operação	OLD	NEW
INSERT	NULL	a linha que está entrando
UPDATE	a linha como estava	a linha como vai ficar
DELETE	a linha que está saindo	NULL

Atenção ao comportamento atual. Desde o PostgreSQL 11, ler NEW num gatilho de DELETE **não dá erro**: NEW vale NULL. O mesmo vale para OLD num INSERT. A função segue em frente com valores vazios, sem aviso — e por isso o engano é difícil de perceber. A mensagem record "new" is not assigned yet, que ainda aparece em tutoriais antigos, era do PostgreSQL 10 ou anterior.

Veja no PostgreSQL. Um gatilho AFTER grava num log o id de OLD e o de NEW:

```
CREATE TABLE log_reserva (
  id          serial PRIMARY KEY,
  operacao    text,
  reserva_old bigint,
  reserva_new bigint
);

CREATE OR REPLACE FUNCTION fn_tg_log_reserva()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO log_reserva (operacao, reserva_old, reserva_new)
  VALUES (TG_OP, OLD.id_reserva, NEW.id_reserva);
  RETURN NULL;
END;
$$;

CREATE TRIGGER tg_reserva_log
  AFTER INSERT OR UPDATE OR DELETE ON reserva
  FOR EACH ROW
  EXECUTE FUNCTION fn_tg_log_reserva();

INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida)
VALUES (3, 301, '2026-11-01', '2026-11-03');
UPDATE reserva SET pessoas = 2 WHERE numero_quarto = 301;
DELETE FROM reserva WHERE numero_quarto = 301;

SELECT operacao, reserva_old, reserva_new FROM log_reserva ORDER BY id;
```

operacao	reserva_old	reserva_new
INSERT		6
UPDATE	6	6
DELETE	6	

Nenhum erro: no INSERT, OLD veio nulo; no DELETE, NEW veio nulo. Para registrar uma exclusão, leia **OLD**.

Outras variáveis prontas: TG_OP ('INSERT', 'UPDATE', 'DELETE'), TG_WHEN ('BEFORE'/'AFTER'), TG_LEVEL ('ROW'/'STATEMENT'), TG_TABLE_NAME. Uma função que atende a vários eventos começa

testando TG_OP.

7.3 Resumo direto: momento, nível e retorno

	BEFORE	AFTER
Quando roda	antes de gravar a linha	depois de gravar
Alterar NEW	tem efeito (corrige o dado)	não tem efeito
Cancelar só a linha com RETURN NULL	sim, em silêncio	não
Vocação	validar e corrigir	registrar e propagar

Nível	Dispara
FOR EACH ROW	uma vez por linha afetada; tem OLD/NEW
FOR EACH STATEMENT	uma vez por comando , mesmo que nenhuma linha seja afetada; é o padrão quando a cláusula é omitida

Tipo de gatilho	O que devolver
BEFORE INSERT / BEFORE UPDATE, por linha	NEW para gravar; NULL cancela a linha
BEFORE DELETE, por linha	OLD para apagar; NULL cancela a linha
AFTER, por linha, e todo FOR EACH STATEMENT	o retorno é ignorado; use RETURN NULL

Executado: um gatilho BEFORE INSERT que faz só RETURN NULL fez o servidor responder INSERT 0 0, sem erro, e a linha não foi gravada. Para **regra de negócio**, isso é ruim: ninguém fica sabendo por que o dado sumiu. Regra violada se comunica com **RAISE EXCEPTION**.

7.4 Exemplo resolvido: um quarto não pode ter reservas com datas sobrepostas

Regra: uma reserva nova não pode ocupar um quarto num período que se sobreponha ao de outra reserva do mesmo quarto. A saída de uma e a entrada da outra podem cair no mesmo dia.

Por que gatilho? O CHECK só enxerga a linha nova; a regra precisa consultar **as outras linhas** da tabela.

Decisões, na ordem em que uma questão de gatilho costuma pedir:

1. **Tabela:** reserva, onde a linha nova entra.
2. **Evento:** INSERT.
3. **Momento:** BEFORE — a verificação acontece antes de gravar, e a linha errada nunca chega à tabela.
4. **Nível:** FOR EACH ROW — cada reserva precisa ser conferida, e só o nível de linha tem NEW.
5. **Teste:** existe outra reserva do mesmo quarto que começa antes da saída nova e termina depois da entrada nova?

```

CREATE OR REPLACE FUNCTION fn_tg_impedir_conflito()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM reserva r
        WHERE r.numero_quarto = NEW.numero_quarto
            AND r.entrada < NEW.saida
            AND NEW.entrada < r.saida
    ) THEN
        RAISE EXCEPTION 'O quarto % já está reservado entre % e %.',
            NEW.numero_quarto, NEW.entrada, NEW.saida
            USING HINT = 'Escolha outro quarto ou outras datas.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_reserva_conflito
BEFORE INSERT ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_tg_impedir_conflito();

```

O quarto 101 está reservado de 03 a 06/07. Testando:

```

INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida)
VALUES (4, 101, '2026-07-05', '2026-07-08');      -- sobrepõe 05/07

INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida)
VALUES (4, 101, '2026-07-06', '2026-07-08');      -- entra no dia da saída

```

```

ERROR:  O quarto 101 já está reservado entre 2026-07-05 e 2026-07-08.
HINT:  Escolha outro quarto ou outras datas.
CONTEXT:  PL/pgSQL function fn_tg_impedir_conflito() line 10 at RAISE

INSERT 0 1

```

7.5 O que acontece quando a regra é violada

Este é o item que mais se esquece ao escrever um gatilho à mão. Quando a função executa o RAISE EXCEPTION:

1. a função para ali mesmo — o RETURN NEW não chega a rodar;
2. o **comando que disparou o gatilho é cancelado por inteiro** — se ele tentava gravar várias linhas, **nenhuma** fica, nem as que estavam certas;
3. **nada daquele comando fica gravado** na tabela;

4. quem executou o comando recebe a mensagem: ERROR (o texto do RAISE), HINT (se houver USING HINT) e CONTEXT (a função e a linha).

Veja o item 2 acontecendo. A tabela tinha 6 reservas; o comando abaixo traz uma linha válida (quarto 301) e uma em conflito (quarto 202, reservado de 01 a 03/08):

```
INSERT INTO reserva (id_hospede, numero_quarto, entrada, saida) VALUES
(3, 301, '2026-10-01', '2026-10-04'),
(4, 202, '2026-08-02', '2026-08-05');
SELECT count(*) FROM reserva;
```

```
ERROR:  0 quarto 202 já está reservado entre 2026-08-02 e 2026-08-05.
HINT:   Escolha outro quarto ou outras datas.
CONTEXT: PL/pgSQL function fn_tg_impedir_conflito() line 10 at RAISE
count
-----
      6
```

A reserva do quarto 301 também não ficou: continuam 6.

7.6 Mais três fatos sobre gatilhos

- **O gatilho vale do momento em que é criado para a frente.** Reservas já gravadas com conflito continuam lá; criar o gatilho não corrige o passado.
- **Gatilhos da mesma tabela e do mesmo evento disparam em ordem alfabética de nome**, não na ordem de criação.
- **Inspecionar e manter:** \d reserva lista os gatilhos da tabela; ALTER TABLE reserva DISABLE TRIGGER tg_reserva_conflito; desliga; DROP TRIGGER tg_reserva_conflito ON reserva; remove (exige o ON tabela).

7.7 Armadilhas comuns

- Declarar parâmetro na função gatilho ou devolver boolean.
- Ler NEW num DELETE achando que ele traz a linha apagada. Vale NULL; use OLD.
- Terminar um BEFORE DELETE com RETURN NEW: como NEW é NULL, isso **cancela a exclusão em silêncio**. Num BEFORE DELETE, devolva OLD.
- Terminar um BEFORE INSERT/UPDATE com RETURN NULL: a linha some sem erro (INSERT 0 0).
- Alterar NEW num gatilho AFTER e esperar que o valor seja gravado.
- Esquecer o FOR EACH ROW: o servidor **não reclama**, o gatilho vira FOR EACH STATEMENT (o padrão) e NEW fica nulo. Executado com a regra da discursiva do simulado: sem FOR EACH ROW, um grupo muito acima do limite foi gravado (INSERT 0 1), sem erro. A regra simplesmente deixa de valer.
- Escrever RAISE NOTICE para barrar a regra: o aviso aparece e o dado **entra**.
- Escrever o CREATE TRIGGER na tabela errada: o gatilho vai na tabela **que recebe o comando** a ser barrado, mesmo que a regra consulte outra.

7.8 Roteiro para escrever um gatilho à mão

1. **Leia a regra e sublinhe** a tabela que recebe o comando, o evento (INSERT, UPDATE, DELETE) e o dado que precisa ser consultado.
2. **Cabeçalho:** `CREATE OR REPLACE FUNCTION fn_tg_...() RETURNS trigger LANGUAGE plpgsql AS $$ — parênteses vazios.`
3. **DECLARE**, se precisar guardar um valor lido de outra tabela (`SELECT ... INTO`).
4. **IF com a regra violada** → `RAISE EXCEPTION 'mensagem';` → `END IF;`
5. `RETURN NEW;` (em INSERT/UPDATE) ou `RETURN OLD;` (em DELETE), e `END; $$;`
6. **CREATE TRIGGER** com momento (BEFORE), evento, ON tabela, **FOR EACH ROW** e **EXECUTE FUNCTION fn_tg_...();**
7. **Explique** por que BEFORE, por que aquele evento e por que FOR EACH ROW.
8. **Explique o efeito da violação** com os quatro pontos da seção 7.5.

7.9 Perguntas de fixação

Pergunta 1. Um gatilho AFTER INSERT ... FOR EACH ROW de auditoria da tabela quarto grava no log o valor de OLD.diaria. O que aparece no log quando um quarto novo é cadastrado?

Confira a resposta: a linha do quarto é gravada e o log recebe a diária **nula**, sem erro nenhum (PostgreSQL 11 ou mais novo). Em INSERT, não há linha antiga: OLD vale NULL. O certo é ler NEW.diaria.

Pergunta 2. Por que a regra da seção 7.4 usa BEFORE, e não AFTER?

Confira a resposta: porque é uma **validação**: conferir antes de gravar faz sentido, e a linha errada nunca chega à tabela. (Um AFTER com RAISE EXCEPTION também cancelaria o comando, mas a vocação do AFTER é registrar e propagar.)

Pergunta 3. Um UPDATE quarto SET diaria = diaria + 50; dispara um gatilho BEFORE UPDATE ... FOR EACH ROW que recusa diária acima de R\$ 400,00. Só o quarto 202 (R\$ 410,00) passaria do teto. Os outros quatro quartos são reajustados?

Confira a resposta: não. Executado: ERROR: Diária de 460.00 acima do teto., e os cinco quartos continuaram com as diárias originais. A exceção cancela o **comando inteiro**, não só a linha que violou.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 8 — Simulado, parte objetiva

Tempo estimado: 20 minutos.

Abra o **simulado**, publicado junto com este roteiro no Classroom, e responda às **questões 1 a 10**.

- Faça **sem consulta**: sem este roteiro, sem os slides, sem internet. O objetivo é descobrir o que você ainda não sabe, e consulta esconde isso.
- Anote a letra de cada questão num papel e **só depois** confira na resolução comentada.
- O simulado **não vale nota**.
- Para cada questão, tente dizer **por que** as outras três alternativas estão erradas. Na prova, os distratores são parecidos com a resposta certa.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 9 — Simulado, questão discursiva à mão

Tempo estimado: 20 minutos.

Responda à **questão 11** do simulado **escrevendo à mão, no papel, com caneta**.

Por que à mão? Porque a prova é **manuscrita**. No papel não há editor que complete a palavra, que feche o parêntese ou que pinte o erro de vermelho. Quem só treinou no computador costuma esquecer, na hora, o `RETURNS trigger`, o `$$`, o `END IF;`, o `FOR EACH ROW` ou o ponto e vírgula. Escrever uma vez à mão, antes da prova, é o treino mais próximo do dia.

Use o roteiro da seção 7.8. Quando terminar, compare a sua resposta com a resposta-modelo e os critérios de correção da resolução comentada, item por item, e dê a sua nota.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Resolução comentada do simulado

Faça o simulado antes de ler esta parte. Se você ler as respostas antes, o simulado perde a função de mostrar o que você ainda não sabe.

Respostas: **1-B · 2-D · 3-A · 4-C · 5-A · 6-D · 7-C · 8-B · 9-D · 10-C**.

Questão 1 — Resposta: B (I e III)

- **I — verdadeira.** Cada linha de permanencia é uma tupla e `hora_entrada` é um atributo; o domínio inclui a regra do negócio (só horários em que o estacionamento funciona), não só o tipo.

- **II — falsa.** NoSQL não é “sempre mais escalável” nem “sempre preferível”: a escolha depende de consistência, consultas, volume, equipe e operação (Etapa 1).
- **III — verdadeira.** A chave estrangeira recusa o valor que não existe na tabela referenciada.
- **IV — falsa.** O MySQL tem MVCC no InnoDB. Afirmar que “não tem” é exatamente o tipo de comparação sem evidência que a Aula 01 condena.
- Por isso: **A** erra ao incluir a II; **C** junta as duas falsas; **D** inclui a IV.

Questão 2 — Resposta: D

A chave é simples (`id_aluguel`), então não há 2FN a violar. `bicicleta_id` → `bicicleta_modelo`, `bicicleta_aro` passa por um atributo que não é chave: dependência transitiva, violação da 3FN. Correção: tabela `bicicleta(id, modelo, aro)`, e `aluguel` guardando só `bicicleta_id`.

- **A — incorreta:** o modelo depende da chave só **através** de `bicicleta_id`; é justamente isso que a 3FN proíbe, e a repetição traz anomalia de atualização.
- **B — incorreta:** não existe “parte da chave” numa chave de uma coluna.
- **C — incorreta:** a mesma bicicleta aparecer em vários alugueis é normal e não viola a 1FN — cada célula continua com um valor só. Proibir isso impediria alugar a bicicleta de novo.

Questão 3 — Resposta: A

O INSERT de várias linhas é um comando só. A terceira linha (`meses = 24`) viola o CHECK (`meses BETWEEN 1 AND 12`), e o comando inteiro é recusado; a tabela continua vazia. Executado:

```
ERROR: new row for relation "plano" violates check constraint
"plano_meses_check"
DETAIL: Failing row contains (3, Bienal, 24, 1990.00).
count
-----
0
```

- **B — incorreta:** não existe gravação parcial num comando que falhou.
- **C — incorreta:** o CHECK não corrige o valor; só aceita ou recusa.
- **D — incorreta:** `id_plano` é identidade e é gerado pelo banco; não precisa ser informado.

Questão 4 — Resposta: C

O pedido exige as salas sem instrutor e os instrutores sem sala: só o FULL OUTER JOIN preserva os dois lados. Com os dados de teste (três salas; um instrutor sem sala), o FULL OUTER JOIN devolveu 4 linhas, incluindo Pilates sem instrutor e Vítor sem sala.

- **A — incorreta:** o LEFT JOIN preserva só as salas; Vítor some.
- **B — incorreta:** o RIGHT JOIN preserva só os instrutores; Pilates some.
- **D — incorreta:** o INNER JOIN perde os dois.

Questão 5 — Resposta: A

As duas são verdadeiras e a II explica a I: o checkpoint marca o ponto a partir do qual a recuperação reprocessa o WAL; por isso não é preciso reproprocessar tudo desde a instalação.

- **B — incorreta:** há relação de causa, sim.
- **C e D — incorretas:** as duas asserções são verdadeiras.

Questão 6 — Resposta: D

Cada unidade guarda **as suas linhas**, com todas as colunas: fragmentação **horizontal**. O relatório que consulta a tabela lógica sem saber onde cada linha está depende de **transparência de localização e de fragmentação**, que é papel do sistema gerenciador.

- **A — incorreta:** vertical separaria colunas, e aqui as colunas são as mesmas.
- **B — incorreta:** replicação seria cada unidade com uma **cópia** de todas as linhas.
- **C — incorreta:** há dados em dois nós; não é centralizado.

Questão 7 — Resposta: C (I e II)

- **I — verdadeira.** Executado: `ERROR: cannot change return type of existing function, com a dica Use DROP FUNCTION fn_preco_banho(text) first.`
- **II — verdadeira.** A função lê a tabela de preços; `STABLE` é a classificação certa, `IMMUTABLE` é para quem depende só dos argumentos.
- **III — falsa.** O código da rotina fica no catálogo do banco (`pg_proc`), não num arquivo do cliente.
- **IV — falsa.** `SELECT ... INTO` sem linha **não dá erro**: a variável fica `NULL`, e a função devolve `NULL`. Executado com um porte inexistente: a coluna voltou vazia.

Questão 8 — Resposta: B

Num `DELETE`, `NEW` vale `NULL`. Em gatilho `BEFORE ... FOR EACH ROW`, devolver `NULL` cancela a linha **sem erro**: por isso o `DELETE 0`. O `INSERT` no arquivo já tinha sido feito e **não** foi desfeito, porque não houve erro. Executado: `DELETE 0`, a atleta continuou em `atleta` e apareceu uma linha em `atleta_arquivo`. Trocando por `RETURN OLD`, o mesmo `DELETE` respondeu `DELETE 1`.

- **A — incorreta:** descreve o comportamento do PostgreSQL 10 ou anterior. Desde o 11, não há erro; `NEW` vale `NULL`.
- **C — incorreta:** `OLD` pode ser lido no `BEFORE DELETE` — tanto que o arquivo recebeu o `id` e o nome da atleta.
- **D — incorreta:** não houve erro para ser tratado; o `INSERT` no arquivo deu certo.

Questão 9 — Resposta: D

O filtro sobre a tabela da direita (`s.data`) está no `WHERE`. Nos filmes sem sessão em setembro, `s.data` é `NULL`, a comparação não é verdadeira e a linha é descartada: o `LEFT JOIN` vira, na prática, um `INNER`

JOIN. Levando a condição para o ON, os filmes sem sessão voltam com a data vazia. Executado com três filmes: no WHERE, voltou 1 linha; no ON, voltaram as 3.

- **A — incorreta:** o LEFT JOIN preserva a tabela da **esquerda** (filme), que já é a certa.
- **B — incorreta:** a comparação de datas funciona; o problema é **onde** ela está.
- **C — incorreta:** trocar para INNER JOIN piora: ele nunca traz filme sem sessão.

Questão 10 — Resposta: C

- **I — verdadeira.** Executado: NOTICE: Nenhum canteiro informado; devolvendo 0. e o resultado 0, sem erro.
- **II — falsa.** Sem tratador, divisão por zero é **erro**. Executado fora da função: ERROR: division by zero. É o bloco EXCEPTION WHEN division_by_zero que transforma o erro em aviso e devolve 0.

Questão 11 — Resposta-modelo

a) Função gatilho

```
CREATE OR REPLACE FUNCTION fn_tg_limite_trilha()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_limite integer;
BEGIN
    SELECT limite_pessoas INTO v_limite
    FROM trilha
    WHERE id = NEW.trilha_id;

    IF NEW.pessoas > v_limite THEN
        RAISE EXCEPTION 'O grupo tem % pessoas, e a trilha % aceita no máximo
%. ',
            NEW.pessoas, NEW.trilha_id, v_limite
        USING HINT = 'Divida o grupo ou escolha outra trilha.';
    END IF;

    RETURN NEW;
END;
$;
```

b) CREATE TRIGGER

```
CREATE TRIGGER tg_grupo_limite
BEFORE INSERT OR UPDATE ON grupo
FOR EACH ROW
EXECUTE FUNCTION fn_tg_limite_trilha();
```

c) Quando dispara. Na tabela grupo, que é onde o grupo é registrado ou alterado. **BEFORE**, porque é uma validação: o limite é conferido antes de gravar, e o grupo grande demais nunca chega à tabela. Eventos **INSERT** (grupo novo) e **UPDATE** (alguém aumenta o número de pessoas depois). **FOR EACH ROW**, porque cada grupo precisa ser conferido e só o nível de linha tem NEW com a quantidade e a trilha.

d) Efeito da violação. O RAISE EXCEPTION interrompe a função; o comando que disparou o gatilho (o INSERT ou o UPDATE) é **cancelado por inteiro** — se trazia várias linhas, nenhuma fica; **nada daquele comando é gravado** em grupo; quem executou recebe a mensagem do erro, com ERROR, HINT e CONTEXT. Executado com uma trilha de limite 8:

```
ERROR:  0 grupo tem 11 pessoas, e a trilha 2 aceita no máximo 8.  
HINT:   Divida o grupo ou escolha outra trilha.  
CONTEXT:  PL/pgSQL function fn_tg_limite_trilha() line 10 at RAISE
```

Um grupo de 6 pessoas na mesma trilha foi aceito (INSERT 0 1); aumentar esse grupo para 9 com UPDATE foi recusado, e o grupo continuou com 6.

Critérios de correção

Item	O que se espera	Pontos
Cabeçalho da função	sem parâmetros, RETURNS trigger, LANGUAGE plpgsql, BEGIN ... END e RETURN NEW	1,0
Lógica da regra	leitura do limite da trilha (SELECT ... INTO ou subconsulta), comparação com NEW.pessoas e RAISE EXCEPTION com mensagem clara	1,5
CREATE TRIGGER	BEFORE, INSERT OR UPDATE, ON grupo, FOR EACH ROW, EXECUTE FUNCTION	1,0
Explicação do disparo	momento, eventos e nível, ligados à regra	0,75
Efeito da violação	função interrompida, comando cancelado por inteiro, nada gravado, mensagem com ERROR, HINT e CONTEXT	0,75
Total		5,0

Também estão certas: a comparação escrita com subconsulta (IF NEW.pessoas > (SELECT limite_pessoas FROM trilha WHERE id = NEW.trilha_id) THEN) e outra mensagem de erro, desde que clara. Perde ponto: RETURN NULL no fim (nenhum grupo seria gravado), RAISE NOTICE no lugar de RAISE EXCEPTION (o grupo entraria), só INSERT no evento (o UPDATE escaparia) e esquecer o FOR EACH ROW.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Referências

DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. **Projeto de banco de dados: uma visão prática**. São Paulo: Érica, 2012.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 11 release notes**. 2018. Disponível em: <https://www.postgresql.org/docs/release/11.0/>. Acesso em: 27 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — PL/pgSQL: SQL Procedural Language**. 2026. Disponível em: <https://www.postgresql.org/docs/17/plpgsql.html>. Acesso em: 27 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — CREATE TRIGGER**. 2026. Disponível em: <https://www.postgresql.org/docs/17/sql-createtrigger.html>. Acesso em: 27 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — Combining queries (UNION, INTERSECT, EXCEPT)**. 2026. Disponível em: <https://www.postgresql.org/docs/17/queries-union.html>. Acesso em: 27 set. 2026.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de gerenciamento de bancos de dados**. 3. ed. Porto Alegre: Grupo A, 2008.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020. E-book. Disponível em: Minha Biblioteca.